

Stock Market Prediction Using Machine Learning

Ayush Nichal¹, Ishan Nalawade², Bhavesh Naranje³, Chaitanya Kapulkar⁴

^{1,2,3,4}Department of Computer Engineering Zeal College of Engineering and Research Pune

Abstract - Predicting stock market behavior remains one of the most challenging problems in applied machine learning due to the volatile, non-stationary, and noisy nature of financial time series. This paper presents an end-to-end web-based system for stock price forecasting and trading signal generation, built using a hybrid machine learning pipeline. Historical Open-High-Low-Close-Volume (OHLCV) data is transformed into a set of scale-invariant technical features, including momentum returns, RSI, MACD percentage, and Bollinger Band position, to address the non-stationarity of raw price series. Three regression models — Random Forest, XGBoost, and a custom Long Short-Term Memory (LSTM) network — are trained and evaluated using walk-forward time-series cross-validation, while a Logistic Regression classifier independently votes on price direction. A BUY or SELL signal is issued only when the best-performing regressor and the classifier agree, and a quality gate based on cross-validated R^2 prevents underperforming models from being deployed for live predictions. The system is implemented using Python, Django, scikit-learn, XGBoost, and PyTorch, with a web interface for configuring forecasts, visualizing results, and tracking historical prediction accuracy. The system was evaluated end-to-end on two instruments — Costco (COST) and the S&P 500 ETF (SPY) — with results showing the LSTM consistently outperforming tree-based models, and a directional classifier that performed near chance level on COST but meaningfully above chance on SPY, highlighting both the system's robustness and the instrument-dependent limits of return predictability.

Key Words: Stock Market Prediction, Machine Learning, LSTM, Random Forest, XGBoost, Time-Series Forecasting

I. INTRODUCTION

Financial markets generate vast amounts of historical price and volume data every trading day, yet most individual investors lack the tools required to systematically analyze this data before making investment decisions. Investment choices are frequently driven by intuition, sentiment, or incomplete information, often resulting in inconsistent outcomes. While institutional investors rely on dedicated

quantitative research teams, retail investors typically lack an equivalent resource.

Stock price movement is influenced by a combination of historical price trends, trading volume, momentum, and broader market sentiment. Manually tracking and interpreting these factors across multiple stocks is both time-consuming and error-prone. This paper proposes a system, "Stock Market Prediction Using Machine Learning," that applies machine learning techniques to historical OHLCV data to forecast future price movement and generate actionable BUY, SELL, or HOLD trading signals through a web-based interface.

Unlike approaches that operate purely as offline research notebooks, the proposed system is deployed as a complete web application, allowing a user to select a stock, configure a forecast horizon, and receive a model-backed signal along with transparent accuracy and backtest metrics. The system further distinguishes itself by combining classical ensemble models (Random Forest, XGBoost) with a deep learning sequence model (LSTM) within a single, quality-gated pipeline, rather than relying on a single algorithm in isolation.

II. LITERATURE REVIEW

Stock market prediction has long been studied due to the volatile, nonlinear, and time-dependent nature of financial data. Traditional statistical approaches such as autoregressive and moving-average models have generally struggled to capture these complexities, motivating a shift toward machine learning (ML) and deep learning (DL) techniques, including Support Vector Machines, Random Forest, k-Nearest Neighbors, Gradient Boosting, Artificial Neural Networks, and Long Short-Term Memory (LSTM) networks [2]–[8].

Among these, LSTM-based models have repeatedly shown strong performance in capturing temporal dependencies in sequential financial data, frequently outperforming classical regression and even other neural architectures in short-term forecasting tasks [1]. At the same time, ensemble tree-based methods such as Random Forest and Gradient Boosting/XGBoost remain competitive baselines for tabular, feature-engineered representations of price data, particularly when engineered technical indicators are used in place of raw price levels.

Systematic reviews of the field further indicate that hybrid and ensemble approaches, combined with careful

preprocessing, feature engineering, and hyperparameter tuning, tend to outperform any single model used in isolation [2], [5], [7]. Persistent challenges noted across this body of work include overfitting on noisy financial series, inconsistent data quality, and the fundamental unpredictability of markets implied by the Efficient Market Hypothesis (EMH). Several researchers have proposed addressing these issues through ensemble methods, alternative data sources such as news sentiment, and explainable AI techniques to improve both predictive accuracy and interpretability [5], [7].

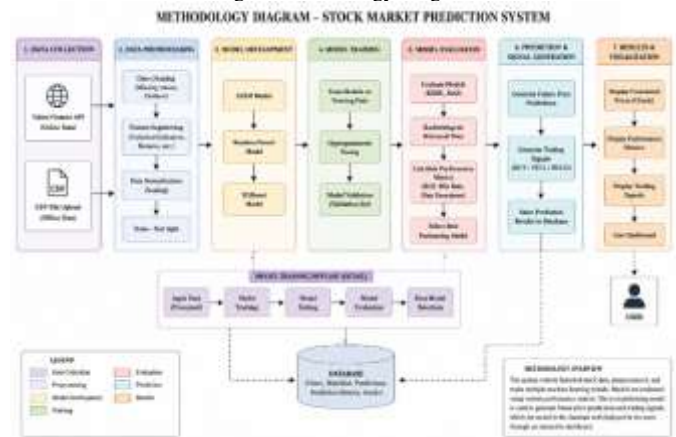
More recent work continues to compare ensemble and deep learning approaches directly. Wu [9] found that Random Forest produced lower RMSE than LSTM on S&P 500 data under low-noise conditions, while Uhunmwangho [11] reported the opposite — XGBoost outperforming LSTM on directional accuracy for Microsoft stock — suggesting that the relative ranking of tree-based and deep learning models is highly dataset- and metric-dependent rather than fixed. Separately, Kashif and Ślepaczuk [10] demonstrated that combining LSTM with a classical time-series model (ARIMA) within a walk-forward optimization framework consistently outperformed either model in isolation across multiple equity indices, reinforcing the case for evaluating multiple models side by side rather than committing to one in advance.

Building on these findings, the present work adopts a multi-model comparison strategy — evaluating Random Forest, XGBoost, and a custom LSTM network using a consistent, walk-forward evaluation methodology — combined with an explicit quality-gating mechanism that, to the best of our knowledge, is not commonly emphasized in prior comparative studies.

III. METHODOLOGY

The proposed system follows a structured machine learning pipeline consisting of data collection, feature engineering, model training and evaluation, quality gating, and inference with signal generation, integrated behind a Django-based web application.

Fig -1: Methodology Diagram



A. Data Collection

Historical OHLCV (Open, High, Low, Close, Volume) data is collected using the yfinance library, which provides reliable access to historical price data for both Indian (NSE) and U.S. equities. The system also supports an offline mode in which users can upload a CSV file containing historical price data, allowing the pipeline to operate independently of live internet connectivity. A minimum number of historical rows is enforced before training is permitted, ensuring that models are not trained on insufficient data.

B. Feature Engineering

Rather than using raw price levels — which are non-stationary and scale-dependent across different stocks and time periods — the system transforms OHLCV data into a set of scale-invariant technical features. These include:

1. Moving-average ratios (price relative to short- and long-term moving averages)
2. Momentum returns over multiple lookback windows
3. Relative Strength Index (RSI)
4. MACD expressed as a percentage of price
5. Bollinger Band width and position
6. Distance from the 200-day moving average (mean-reversion indicator)

This scale-invariant design allows a single feature engineering pipeline to be reused across stocks of very different price magnitudes and volatility profiles without retraining the feature logic itself.

C. Model Development

Three candidate regression models are trained to forecast future returns:

1. *Random Forest Regressor* — an ensemble of decision trees trained on engineered features.

2. *XGBoost Regressor* — a gradient-boosted tree ensemble offering strong performance on tabular financial features with built-in regularization.
3. *Long Short-Term Memory (LSTM) Network* — a custom two-layer recurrent neural network implemented in PyTorch, performing direct multi-step forecasting of future daily returns from sequences of engineered features.

In parallel, a *Logistic Regression classifier* is trained to predict the direction (up/down) of the next price movement, independent of the regression models.

All models are evaluated using walk-forward time-series cross-validation with an explicit leakage gap between training and validation folds, rather than a single random train-test split. This better reflects the real-world deployment scenario in which a model must generalize to genuinely unseen future data. Feature scaling is performed independently within each fold to prevent information leakage from validation data into the training process. This walk-forward methodology follows established practice in recent algorithmic trading literature, where standard k-fold cross-validation has been shown to be unsuitable for financial time series due to the violation of the independent-and-identically-distributed assumption [10].

D. Quality Gating

Before any trained model is persisted for live use, its cross-validated R^2 score is checked against a minimum quality threshold of -0.5. Models that fail to meet this threshold are not saved, preventing unreliable models from being deployed for live predictions. This step reflects an explicit acknowledgment that, consistent with the Efficient Market Hypothesis, predictive R^2 for financial returns is typically low, and a deployed model must still be held to a measurable minimum standard rather than being deployed unconditionally.

E. Inference and Signal Generation

At prediction time, the best-performing model — selected by lowest validation RMSE during training — generates a forecasted price path over the user-specified horizon (1–30 days). Independently, the Logistic Regression classifier votes on the expected direction of price movement using the latest feature row. A BUY or SELL signal is issued only when the regressor's projected price change and the classifier's directional vote agree; otherwise, the system returns a HOLD signal. This agreement-based approach is intended to reduce the rate of false signals that might arise from relying on a single model in isolation.

F. Backtesting

For every trained model, the pipeline simulates a simple trading strategy based on the classifier's historical signals on the held-out test set, computing Return on Investment (ROI), win rate, and maximum drawdown. These backtest metrics, along with standard regression error metrics (MAE, RMSE), are surfaced to the user alongside every prediction, providing transparency into model reliability rather than presenting forecasts as unconditionally accurate.

G. Web Application

The complete pipeline is exposed through a Django-based web application. Users can register, build a personalized stock watchlist (including bulk import from curated indices such as Nifty 50 and S&P 500), configure a forecast (stock, horizon, data source), and view the resulting forecast chart, signal, and metrics. Administrators can trigger model training or retraining for individual stocks or in bulk across the entire tracked universe, and can monitor platform-wide usage and prediction history through a dedicated dashboard.

IV. RESULTS

To evaluate the proposed system, the complete pipeline was executed end-to-end on two instruments with very different characteristics: Costco Wholesale Corporation (COST), a single large-cap equity, and the SPDR S&P 500 ETF Trust (SPY), a diversified broad-market index fund. Both were evaluated using ten years of daily OHLCV data retrieved via the yfinance API. After feature engineering, 2,263 valid rows remained for COST and 2,261 for SPY (from 2,514 and 2,512 raw trading days respectively), with the small difference attributable to rows lost to rolling-window indicator calculations. Each dataset was split chronologically — COST into 1,810 training and 453 test rows, SPY into 1,808 training and 453 test rows — consistent with the walk-forward validation strategy described in Section III-C.

A. Model Comparison

TABLE I. WALK-FORWARD CV RESULTS, COST VS. SPY (10-YEAR HISTORY)

Model	COST MAE	COST RMSE	COST R^2	SPY MAE	SPY RMSE	SPY R^2
Random Forest	1.9111	2.5837	-0.1285 ± 0.0606	1.5762	2.1284	-0.1486 ± 0.0319
XGBoost	1.9680	2.6540	-0.1973 ± 0.0628	1.6315	2.2068	-0.2398 ± 0.0533
LSTM	1.8053	2.4646	-0.0103 ± 0.0130	1.4569	2.0423	-0.0103 ± 0.0129

The LSTM network achieved the lowest MAE and RMSE on both instruments and was automatically selected by the pipeline as the best-performing regressor

in each case. Notably, the LSTM's cross-validated R^2 was nearly identical for COST (-0.0103) and SPY (-0.0103) despite the two instruments having different price levels, volatility, and composition — suggesting that, for this architecture and feature set, the LSTM converges toward a similar small-negative- R^2 regime regardless of the underlying instrument. Both Random Forest and XGBoost recorded consistently negative R^2 scores on both instruments, with XGBoost performing worst in every case. Both LSTM models passed the system's quality gate (minimum R^2 threshold of -0.5) and were approved for live deployment; the tree-based models, while trained and saved, were not selected as the best regressor for either stock.

B. Classifier and Backtest Performance

The Logistic Regression classifier, trained independently on the same engineered features to predict price direction, produced markedly different results on the two instruments, as shown in TABLE II.

TABLE II. DIRECTIONAL CLASSIFIER AND BACKTEST PERFORMANCE, COST VS. SPY

Metric	COST	SPY
Accuracy	48.22%	50.89%
Precision	48.91%	58.28%
Recall	49.12%	64.94%
F1-Score	49.02%	61.43%
Backtest ROI	0.03%	14.24%
Backtest Win Rate	48.91%	54.64%
Simulated Trades	229	302

This is the most notable contrast in the results: while the COST classifier performed at essentially chance level (48.22% accuracy, 0.03% backtest ROI), the SPY classifier performed meaningfully better across every metric, with a 14.24% backtest ROI over 302 simulated trades and a win rate of 54.64% — nearly 5 percentage points of accuracy above chance, and a recall of 64.94% indicating it correctly flagged most genuine upward moves. Despite both regressors converging to similar R^2 values (Section IV-A), the directional classifier — a separate model trained on the same features — found a measurably stronger signal on the broad-market ETF than on the single equity.

C. Live Inference and Signal Generation

Following training, the system performed live inference for a 10-day forecast on both instruments, producing opposite signals on the same day. TABLE III summarizes both outcomes.

TABLE III. LIVE INFERENCE AND SIGNAL OUTPUT, COST VS. SPY

	COST	SPY
Current Price	\$951.45	\$744.39
10-Day Forecast	\$922.14	\$773.52
Projected Change	-3.08%	+3.91%

	COST	SPY
Classifier Vote	DOWN (63.20%)	UP (54.97%)
Signal	SELL	BUY

For COST, the LSTM projected a 3.08% price decline and the classifier independently voted DOWN with 63.20% confidence; because both agreed on direction, the system issued a SELL signal (Fig-2, Fig-3). For SPY, the LSTM projected a 3.91% price increase and the classifier voted UP with 54.97% confidence; with regressor and classifier again in agreement, the system issued a BUY signal (Fig-5). In both cases, the forecast result page displayed the signal alongside the model used (LSTMForecaster in both instances), the forecast horizon, and the model's training-time MAE and RMSE.

Fig -2: COST forecast result page showing predicted price, signal, and error metrics



Fig -3: COST price forecast chart — actual close, model fit, and forward forecast



D. System Behavior Across Multiple Stocks

Fig-3 shows the administrative Stock Management interface, confirming the platform was actively tracking four equities at the time of testing: AAPL, COST, NVDA, and SPY — spanning individual large-cap equities as well as a broad-market index fund. The successful training and inference runs on both COST and SPY, using the identical pipeline and feature logic, demonstrate that the system generalizes across instruments of very different price levels (COST near \$950, SPY near \$744) and composition (single stock vs. diversified fund) without requiring per-stock changes to the underlying code.

Fig -4: Administrative Stock Management interface showing tracked tickers



Fig -5: SPY price forecast chart generated using the identical pipeline



E. Automatic Training and Quality Control in Action

The system logs captured two independent instances of the auto-retrain mechanism described in Section III: when predictions were requested for COST and, separately, for SPY, and no trained artifacts were found on disk for either, the system automatically triggered the full training pipeline — data download, feature engineering, walk-forward evaluation, quality gating, and artifact persistence — before retrying and completing the original prediction request in both cases, without manual intervention. The SPY run completed in approximately two minutes, comparable to the COST run.

V. DISCUSSION

The side-by-side results for COST and SPY illustrate both the consistency and the instrument-dependent variability of the proposed system.

A. Model Comparison

The LSTM network outperformed both Random Forest and XGBoost on every error metric for both instruments, aligning with one strand of the comparative literature reviewed in Section II — for instance, the hybrid LSTM-ARIMA results of Kashif and Ślepaczuk [10], which found LSTM-based approaches outperforming alternatives across multiple equity indices under a similar walk-forward framework. It does not, however, align universally with all prior findings: Wu [9] reported the opposite ranking (Random Forest outperforming

LSTM) on S&P 500 data, and Uhunmwangho [11] found XGBoost superior to LSTM on Microsoft stock. The fact that the present study's two instruments produced near-identical LSTM R^2 values (-0.0103 in both cases) while the tree-based models diverged more (Random Forest: -0.1285 vs. -0.1486 ; XGBoost: -0.1973 vs. -0.2398) suggests the LSTM architecture may generalize more uniformly across instruments than the tree-based alternatives in this feature space, though confirming this would require testing across a larger and more diverse set of tickers than the two examined here.

B. Negative R^2 for Tree-Based Models

Both tree-based models produced negative cross-validated R^2 on both COST and SPY, meaning that predicting the historical mean return would, on average, have outperformed either model on unseen data in both cases. This is consistent with the quality-gating philosophy described in Section III-D: rather than treating this as a system failure, it reflects the well-documented difficulty of forecasting daily-level equity and index returns. Had the LSTM also failed to clear the -0.5 R^2 threshold on either instrument, the system would have declined to deploy a model rather than serve an unreliable forecast — the quality gate functioned as intended in both runs.

C. The COST/SPY Classifier Divergence

The most striking finding in this evaluation is the gap between the two instruments' directional classifiers: near-chance performance on COST (48.22% accuracy, 0.03% ROI) versus a measurably better-than-chance result on SPY (50.89% accuracy, 14.24% ROI). Because both instruments used the identical feature set, model architecture, and training procedure, this difference is best attributed to the underlying instruments themselves rather than to the system. SPY, as a diversified index fund, aggregates the price movements of 500 underlying companies, which tends to smooth out single-company idiosyncratic noise (earnings surprises, company-specific news) that often dominates short-term price action for an individual stock like COST. This is consistent with the broader literature's observation that technical-indicator-based directional prediction tends to perform more consistently on diversified instruments than on individual equities [9], [11], and offers a concrete illustration of why the proposed architecture surfaces per-stock backtest metrics to the user rather than presenting a uniform claim of reliability across all tracked tickers.

D. Signal Generation in Practice

Both live inference examples demonstrate the intended behavior of the agreement-based signal logic described in Section III-E: for COST, the LSTM's projected

decline and the classifier's DOWN vote agreed, producing SELL; for SPY, the LSTM's projected increase and the classifier's UP vote agreed, producing BUY. In both cases, agreement between the two independently trained models was required before an actionable signal was issued, rather than relying on either model in isolation.

E. Cross-Stock Generalization

The system trained and served forecasts for both a single high-priced equity (COST, ~\$951) and a lower-priced, broad-market ETF (SPY, ~\$744) using identical code and feature logic, supporting the claim made in Section III-B that the scale-invariant feature engineering approach generalizes across instruments without per-stock customization. This structural property held even though the two instruments produced very different classifier and backtest outcomes, indicating that generalization of the pipeline and predictive accuracy of any individual model are separate properties — the former held consistently across both runs, while the latter varied considerably.

F. Limitations

This evaluation covers two of the four tickers currently tracked by the system (COST and SPY); AAPL and NVDA were tracked in the platform (Fig-3) but were not exercised through a complete training run captured in this study. A more comprehensive evaluation extending Tables I–III to all four tickers, and ideally to a larger universe of stocks spanning multiple sectors, would allow firmer conclusions about whether the COST/SPY classifier divergence generalizes into a broader pattern or is specific to these two tickers. Additionally, the backtest results reported here do not account for transaction costs, slippage, or taxes, which would reduce the net ROI in a live trading scenario.

VI. CONCLUSION

This paper presented a complete machine learning pipeline for stock price forecasting and trading signal generation, combining classical ensemble models (Random Forest, XGBoost) with a deep learning sequence model (LSTM) and a Logistic Regression classifier within a single, quality-gated system. Scale-invariant feature engineering, walk-forward cross-validation, and an explicit minimum-quality threshold collectively address common pitfalls in financial machine learning, including non-stationarity, data leakage, and the deployment of unreliable models.

End-to-end evaluation on two instruments — COST and SPY — showed the LSTM consistently outperforming both tree-based models on regression error metrics, while the directional classifier and backtest results

diverged sharply between the two instruments: near-chance performance on the individual equity (COST) versus a meaningfully profitable backtest on the diversified index fund (SPY). This finding underscores that the reliability of machine-learning-driven trading signals is not uniform across instruments, even when the underlying model architecture and feature pipeline are held constant — reinforcing the value of the system's transparent, per-stock accuracy and backtest reporting over a one-size-fits-all reliability claim.

As with all approaches to stock price prediction, the system's predictive power remains bounded by the inherent unpredictability of financial markets; no model in this study, nor in the broader literature, eliminates this fundamental limitation. Future work includes extending the evaluation to a larger universe of stocks across multiple sectors, integrating sentiment analysis from news and social media as an additional model input, extending the architecture to Transformer-based sequence models, supporting brokerage API integration for direct trade execution, and accounting for transaction costs and slippage in the backtest methodology.

2. Body of Paper

The body of the paper consists of numbered sections that present the main findings. These sections should be organized to best present the material.

It is often important to refer back (or forward) to specific sections. Such references are made by indicating the section number, for example, “In Sec. 2 we showed...” or “Section 2.1 contained a description...” If the word Section, Reference, Equation, or Figure starts a sentence, it is spelled out. When occurring in the middle of a sentence, these words are abbreviated Sec., Ref., Eq., and Fig.

At the first occurrence of an acronym, spell it out followed by the acronym in parentheses, e.g., charge-coupled diode (CCD).

VII. REFERENCES

- [1] M. I. K. Javed, "Stock Market Price Prediction using Machine Learning Techniques," *American International Journal of Sciences and Engineering Research*, vol. 7, no. 1, Feb. 2024, doi: 10.46545/aijsr.v7i1.308.
- [2] L. N. Mintarya, J. N. M. Halim, C. Angie, S. Achmad, and A. Kurniawan, "Machine Learning Approaches in Stock Market Prediction: A Systematic Literature Review," *Procedia Computer Science*, vol. 216, pp. 96–102, 2023.
- [3] D. Liu, A. Chen, and J. Wu, "Research on Stock Price Prediction Method Based on Deep Learning," in *Proc. 2nd Int. Conf. Information Technology and Computer Application (ITCA)*, 2020, pp. 69–73, doi: 10.1109/ITCA52113.2020.00022.

[4] G. Bathla, "Stock Price Prediction Using LSTM and SVR," in *Proc. 6th Int. Conf. Parallel, Distributed and Grid Computing (PDGC)*, 2020, pp. 211–214, doi: 10.1109/PDGC50313.2020.9315800.

[5] M. Bansal, A. Goyal, and A. Choudhary, "Stock Market Prediction with High Accuracy Using Machine Learning Techniques," *Procedia Computer Science*, vol. 215, pp. 247–265, 2022.

[6] Md. M. Akhtar, A. S. Zamani, S. Khan, A. S. A. Shatat, S. Dilshad, and F. Samdani, "Stock Market Prediction Based on Statistical Data Using Machine Learning Algorithms," *Journal of King Saud University – Science*, vol. 34, no. 4, art. 101940, 2022.

[7] R. Najem, M. Fakhouri Amr, A. Bahnasse, and M. Talea, "Advancements in Artificial Intelligence and Machine Learning for Stock Market Prediction: A Comprehensive Analysis of Techniques and Case Studies," *Procedia Computer Science*, vol. 231, pp. 198–204, 2024.

[8] A. Subasi, F. Amir, K. Bagedo, A. Shams, and A. Sarirete, "Stock Market Prediction Using Machine Learning," *Procedia Computer Science*, vol. 194, pp. 173–179, 2021.

[9] H. Wu, "Comparison of Random Forest and LSTM in Stock Prediction," *Advances in Economics, Management and Political Sciences*, vol. 86, no. 1, pp. 28–34, May 2024.

[10] K. Kashif and R. Ślepaczuk, "LSTM-ARIMA as a Hybrid Approach in Algorithmic Investment Strategies," arXiv:2406.18206, Jun. 2024.

[11] R. Uhunmwangho, "Comparing XGBoost and LSTM Models for Prediction of Stock Price Direction," *Mountain Top University Journal of Applied Science and Technology (MUJAST)*, vol. 4, no. 2, pp. 64–88, 2024.

[12] M. Saberironaghi, J. Ren, and A. Saberironaghi, "Stock Market Prediction Using Machine Learning and Deep Learning Techniques: A Review," *AppliedMath*, vol. 5, no. 3, art. 76, 2025, doi: 10.3390/appliedmath5030076.